

2026 年 4 月 28 日 Version x.x

ハッカソン ～計画書と設計書～

チーム 23

24G1038 : 片岡 聖

24G1175 : 地曳 賢人

24G1131 : 御代川 稜

25G1021 : 梅澤 颯太

25G1053 : 齋藤 翔太

25G1065 : 塩澤 匠生

目次

第 1 章	プロジェクトの計画書	1
1.1	プロジェクトの概要	1
1.1.1	プロジェクトの題目	1
1.1.2	プロジェクトの目的	1
1.1.3	既存の技術とプロジェクトの独自性	2
1.2	スコープ・マネジメント	4
1.2.1	成果物	4
1.2.2	プロジェクトの対象範囲と非対象範囲	4
1.2.3	機能分解構造 FBS	4
1.2.4	製品分解構造 PBS	4
1.2.5	FBS と PBS の対応関係	4
1.3	作業計画	4
1.3.1	作業分解構造 WBS と管理番号	4
1.3.2	アローダイアグラムとクリティカルパス	4
1.3.3	役割分担	4
1.3.4	ガントチャート	4
1.4	検証と妥当性確認: V&V 計画	4
1.4.1	プロジェクトの成果物に対する有効指標 MOE	4
1.4.2	有効指標 MOE に対応する性能指標 MOP	4
1.4.3	システムテストで評価する性能指標 MOP	4
1.4.4	結合テストで評価する性能指標 MOP	4
1.4.5	単体テストで評価する性能指標 MOP	4
1.4.6	プロジェクト中に監視する技術性能指標 TPM	4
1.4.7	プロジェクト中に監視する指標 TPM	4
1.5	資源・調達管理	4
1.6	リスク管理	4
第 2 章	システムの基本設計	5

2.1	機能一覧	5
2.2	システム構成図	6
2.3	状態遷移図	6
第3章	システムの詳細設計	7
3.1	部品一覧	7
3.2	ソフトウェア設計	8
3.2.1	関数設計	8
3.3	テストの設計	8
3.3.1	単体テスト	8
3.4	AI 使用について	9

第 1 章 プロジェクトの計画書

1.1 プロジェクトの概要

本プロジェクトでは、13 週をかけてハードウェアとソフトウェアを組み合わせたシステムを Arduino を用いてオーケストラを目標とする。23 班では指揮によるテンポ変化システムを開発する。その際に、テンポや音量を変化できるようにする。

1.1.1 プロジェクトの題目

指揮用 Arduino から指揮をするオーケストラ。

1.1.2 プロジェクトの目的

プロジェクトの目的と最終的なゴール、解決すべき課題を記述する。本プロジェクトの目的は、時間帯や天候、ネットワーク環境などの外部条件に左右されず、明るさなどの制約も受けない環境で、音楽経験のない人でも直感的な指揮動作だけで演奏に参加できるシステムを開発することである。具体的には、指揮棒を振る周期を検出して楽曲のテンポへ反映し、さらに加速度の大きさに応じた音量変化や LED による視覚的演出を加えることで、誰でも簡単に楽しめる体験型電子オーケストラの実現を目指す。最終的なゴールは、指揮棒を振った際に、振りの大きさや周期の速度によって音楽の音量の大きさやテンポが変化したならば完成である。課題としては以下のような観点から考えられる。

- ・ 指揮動作を正確に検出する課題がある。なぜなら、指揮棒の振りには個人差があり、速さ・大きさ・角度も異なるため、さまざまな振り方でも安定して動作を検出する必要がある。
- ・ テンポへリアルタイム反映する課題がある。なぜなら、指揮棒を振った周期に合わせて、音楽のテンポを遅延なく自然に変化させる必要がある。反応が遅いと操作感が悪くなる。

- ・誤検出を防ぐ課題がある。なぜなら、小さな揺れや不要な動きを誤って指揮動作と認識すると、テンポが乱れるため、必要な動作だけを判定する仕組みが必要である。
- ・音楽経験がない人でも使いやすくする課題がある。なぜなら、専門知識がなくても直感的に使えるように、簡単な操作方法と分かりやすいフィードバックが必要である。
- ・視覚的に状態を伝える課題がある。なぜなら、現在のテンポや音量変化が分かりにくいと利用者が操作しづらいため、LED 表示などで演奏状態を可視化する必要がある。
- ・環境条件に左右されない課題がある。なぜなら、明るさ、時間帯、天候、通信環境などに依存せず、屋内外で安定して使用できるシステムにする必要がある。

1.1.3 既存の技術とプロジェクトの独自性

既存技術として、テルミンのような非接触型電子楽器や、カメラ・センサを用いて身体動作を音楽表現へ変換するシステムが存在する [1]。また、Wii Music[2] では、コントローラの動作から拍を検出し、音楽のテンポを制御している。

しかし、これらのシステムは高い演奏技術を必要としたり、明暗などの環境条件に影響されやすいという課題がある。さらに、多くは 1 台の機器内で音源生成を行う構成であり、複数端末による分散演奏や、リアルタイムな指揮制御には十分対応していない。

以上より、複数マイコンによる分散演奏と、身体動作によるリアルタイム指揮制御を両立するシステムには新規性がある。

1.2 スコープ・マネジメント

1.2.1 成果物

1.2.2 プロジェクトの対象範囲と非対象範囲

1.2.3 機能分解構造 FBS

1.2.4 製品分解構造 PBS

1.2.5 FBS と PBS の対応関係

1.3 作業計画

1.3.1 作業分解構造 WBS と管理番号

1.3.2 アローダイアグラムとクリティカルパス

1.3.3 役割分担

1.3.4 ガントチャート

1.4 検証と妥当性確認: V&V 計画

1.4.1 プロジェクトの成果物に対する有効指標 MOE

1.4.2 有効指標 MOE に対応する性能指標 MOP

1.4.3 システムテストで評価する性能指標 MOP

1.4.4 結合テストで評価する性能指標 MOP

1.4.5 単体テストで評価する性能指標 MOP

第 2 章 システムの基本設計

設計書は作成する成果物によって変化します。適宜修正して利用してください。

2.1 機能一覧

機能分解構造 FBS の要素の説明

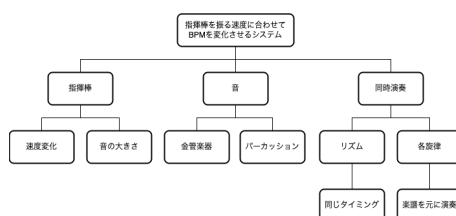


図 2.1 本プロジェクトの FBS 図

2.1 は、本プロジェクトの FBS である。本システムの最上位機能は、「指揮棒を振る速度に合わせて BPM を変更するシステム」である。この目的を実現するために、機能を以下の要素へ分解した。

まず、「演奏」はシステム全体の出力機能であり、指揮者の操作に応じて複数の楽器パートを再生する役割を持つ。その下位要素として「楽器」があり、演奏音を構成する音源群を表す。さらに、楽器は「パーカッション」「金管楽器」「木管楽器」に分類され、各パートごとに異なる音色や役割を担当する。

次に、「指揮棒」は入力インターフェースであり、利用者の身体動作を取得する装置である。指揮棒の振り方に応じて、「速度変更」および「音の大きさ」の制御を行う。速度変更は、指揮棒を振る周期や速さから楽曲の BPM を変化させる機能である。音の大きさは、加速度の強さや振り幅に応じて音量を調整する機能である。

また、演奏される楽曲は「旋律」により構成される。旋律はさらに「主旋律」と「リズム」に分かれ、主旋律が楽曲の中心的なメロディを担当し、リズムがテンポ感や拍子を形成する。

2.2 システム構成図

ハードウェア，ソフトウェアなどの構成図を記載する。

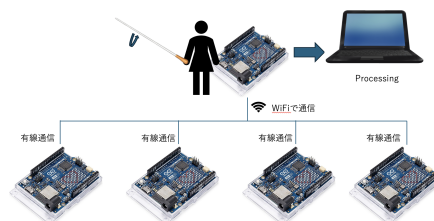


図 2.2 本プロジェクトのソフトウェアの構成図

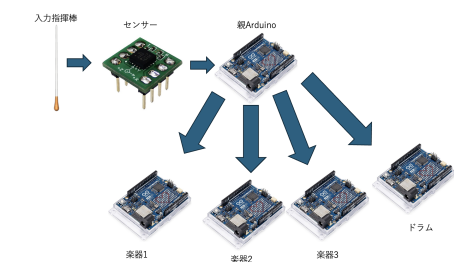


図 2.3 本プロジェクトのハードウェアの構成図

2.2 は本プロジェクトのソフトウェアの概念図である。2.3 は本プロジェクトのハードウェアの構成図である

2.3 状態遷移図

2.4 は本プロジェクトの状態遷移図である。

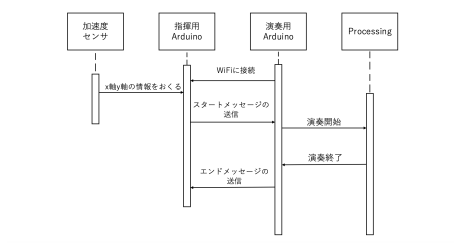


図 2.4 本プロジェクトのハードウェアの状態遷移図

第 3 章 システムの詳細設計

設計書は作成する成果物によって変化します。適宜修正して利用してください。

3.1 部品一覧

製品分解構造 PBS の要素の説明

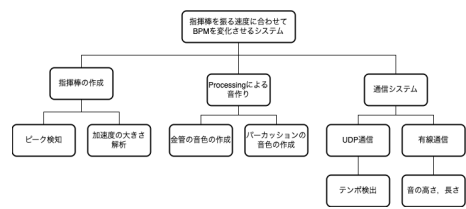


図 3.1 本プロジェクトの PBS 図

2.1 は本プロジェクトの PBS 図である。

表 3.1 必要機材一覧

機材名	数量	型番
Arduino UNO R4 WiFi	5 台	ABX00087
3 軸加速度センサ	1 個	KXR94-2050
スピーカー	1 個	8Ω 1W 丸形スピーカー (20-A GB)
Type-C to C ケーブル	4 本	USB Type-C to Type-C 1.0m
PC	1 台	Mac

3.2 ソフトウェア設計

本システムでは、保守性・再利用性・役割分担の明確化を目的として、入力・ロジック・出力の 3 フェーズ構成でソフトウェアを設計する。各機能は独立したモジュールとして実装し、共有データを介して連携する。また、周期処理にはタイマ制御を用い、`delay()` によるブロッキング処理は行わない。

3.2.1 関数設計

関数名	機能
<code>setup()</code>	初期化处理
<code>loop()</code>	入力・処理・出力の繰返し実行
<code>SensorModule:update()</code>	加速度センサ値取得
<code>TempoModule:update()</code>	振り周期から BPM 算出
<code>NetworkModule:update()</code>	BPM 情報送受信
<code>SoundModule:update()</code>	音楽再生制御
<code>LedModule:update()</code>	LED 表示更新
<code>ModuleTimer:isReady()</code>	周期実行判定

3.3 テストの設計

どのようにテストを行うのか、例えば、テストデータの設計などを記述。

本システムでは、保守性・再利用性・役割分担の明確化を目的として、ソフトウェアを入力・ロジック・出力の 3 フェーズに分割して設計している。このため、各モジュール単位での単体テストと、モジュール間の結合テスト、およびシステムテストを段階的に実施する。

3.3.1 単体テスト

単体テストは以下のようにする。

- ・入力モジュールのテストは以下になる。入力モジュールに対しては、模擬的な入力データを用いて、データ取得および整形処理が正しく行われるかを確認する。テストデータとしては、通常動作を想定した正常値に加え、最小値・最大

値といった境界値、および範囲外の異常値を用いる。これにより、様々な入力条件下での動作の妥当性を検証する。

- ・ ロジックモジュールについては以下ようになる。ロジックモジュールに対しては、入力データに対する処理結果が期待通りであるかを確認する。具体的には、入力と期待出力をあらかじめ定義し、実際の出力と比較することで検証を行う。また、条件分岐を含む処理については、各分岐がすべて実行されるようにテストデータを設計し、網羅的なテストを行う。
- ・ 出力モジュールは以下ようになる。出力モジュールに対しては、ロジックモジュールの出力を入力として与え、正しい形式およびタイミングで出力が行われるかを確認する。表示内容や通信データについてはログを取得し、その内容を検証する。

3.3.2 結合テスト

結合テストは以下ようになる。各モジュールを接続し、入力から出力までの一連の処理が正しく動作するかを確認する。特に、モジュール間で受け渡されるデータの形式や値に不整合がないかを重点的に検証する。

3.3.3 タイマ制御のテスト

タイマ制御のテストは以下ようになる。本システムでは周期処理にタイマ制御を用い、`delay()` によるブロッキング処理を行わない設計としている。そのため、処理が所定の周期で実行されるかを確認する。具体的には、タイムスタンプを用いて処理間隔を測定し、周期のばらつきや遅延の有無を評価する。また、他処理と並行して動作することも確認する。

3.3.4 システムテスト

システムテストは以下ようになる。最終的に実環境においてシステム全体の動作確認を行う。長時間の連続動作による安定性の確認や、入力異常などを想定した異常系テストを実施し、実運用に耐えうることを検証する。

3.4 AI 使用について

文章構成, 頭の中のアイデアを箇条書きにし, その内容の要約,、や。や語尾の統一のために使用.

参考文献

- [1] 川合雄佑、伊藤彰教” 非接触型電子楽器の開発を通じたサウンドデザイン東京工科大学メディア学部, 先端芸術音楽創作学会会報 Vol.14 No. 1 pp.14 19,2022
- [2] 任天堂株式会社.,Wii music 公式サイト, <https://www.nintendo.co.jp/wii/r64j/index.html>, 2008. 2026-04-28 閲覧.